# Trabalho 2 — Comparando Funções Hash

## 1. Identificação

O grupo implementa quatro funções hash, aplica cada uma a quatro conjuntos de 700 chaves em uma tabela de 1000 posições com rehashing linear e compara a distribuição das chaves e o número de colisões.

| Campo | Informação |
|---|---|
| Trabalho | Nº 2 — Comparando funções hash |
| Assunto | Tabela hash: qualidade da função hash e distribuição das chaves |
| Disciplina | Resolução de problemas estruturados em computação |
| Grupo | Grupo de 3 ou 4 acadêmicos; trabalho individual não é permitido (item 3.7). O trabalho do grupo é definido por sorteio em sala. |
| Linguagem | Java, somente com os recursos autorizados na seção 3 |
| Prazo final | 06/11/2026 (sexta-feira), às 23:59 (horário de Brasília) |
| Nota | De 0 a 100 pontos (rubrica na seção 9) |
| Entregas | `Trabalho2.java`, `resultados_trabalho2.csv` e `relatorio_trabalho2.pdf` |

## 2. Objetivo

Medir como a escolha da função hash muda a distribuição das chaves na tabela e o custo de inserir e buscar, usando chaves aleatórias e chaves com padrões "viciados".

**Base teórica (slides de Tabela Hash).**

- Uma **função hash** transforma uma chave em um índice válido da tabela. No exemplo dos slides, `h(chave) = chave % 1000` devolve os três últimos dígitos: `h(9837561) = 561`.
- Chaves diferentes podem resultar na mesma posição (**colisão**). Aqui as colisões são tratadas por rehashing linear, `r(i) = (i + 1) % tamanho`.
- Quanto menor o número de colisões, melhor a função, porque as chaves se espalham de maneira uniforme e economizam comparações.

**O que o grupo deve fazer.**

1. Implementar as 4 funções hash definidas na seção 4.
2. Gerar 4 conjuntos de 700 chaves (seção 5) e aplicar cada função a cada conjunto: 16 combinações.
3. Coletar as métricas da seção 5, validar com os testes da seção 6 e entregar o relatório da seção 7.

**Fator de carga:** em todas as combinações são 700 chaves em 1000 posições, ou seja, 70%.

## 3. Regras gerais

### 3.1 Proibição de inteligência artificial generativa

- É proibido usar qualquer IA generativa em qualquer etapa: escrever ou corrigir código, gerar comentários, explicar algoritmos, escrever, revisar ou traduzir o relatório. Isso inclui assistentes de conversa (ChatGPT, Gemini, Claude, Copilot, DeepSeek, Grok e semelhantes) e o autocompletar com IA de editores e IDEs.

- Se for constatado uso de IA generativa, **todos os integrantes do grupo ficam com nota 0 (zero) neste trabalho**, sem reposição.
- Cópia de outro grupo, de repositórios ou de sites tem a mesma consequência (nota 0) para todos os grupos envolvidos.
- O professor pode pedir a qualquer integrante que explique, oralmente ou por escrito, qualquer trecho do código ou do relatório. Não saber explicar o próprio trabalho é tratado como indício de uso de IA ou de cópia.
- É permitido consultar os slides da aula, livros-texto e a documentação oficial do Java, somente para os recursos autorizados em 3.2.

## 3.2 Recursos de Java autorizados

Somente os itens abaixo podem aparecer no código. **Tudo que não está nesta lista é proibido.**

- **Tipos:** `int`, `float`, `long`, `boolean`, `String`.
- **Vetores:** unidimensionais (`int[]`), criados com `new`, inicializados com `{...}` ou preenchidos com `for`; atributo `.length`.
- **Estruturas de controle:** `for`, `while`, `do while`, `if`, `else`, `else if`.
- **Operadores:** aritméticos (`+ - * / %`), `++`, `--`, atribuição (`=`, `+=`, `-=`), comparação (`== != < > <= >=`), lógicos (`&&`, `!` e o "ou" lógico de dois traços verticais) e conversão de tipo `(int)`, `(float)`, `(long)`.
- **Métodos:** métodos `static` com parâmetros e `return`; variáveis `static` declaradas na classe principal. Recursão é permitida, mas não é necessária.
- **Classes:** somente a classe principal `Trabalho2`. Nenhuma outra.
- **Sorteio:** `java.util.Random`, sempre com a semente 1990 (item 3.5).
- **Saída no console:** `System.out.print`, `System.out.println`, `System.out.printf` e `String.format`.
- **Métricas:** `System.nanoTime()`.
- **Arquivo CSV:** `java.io.PrintWriter` (métodos `println` e `close`) e `java.io.IOException`. **Única exceção à proibição de `throws`:** escrever `throws IOException` na assinatura de `main`, que é o único método que cria o `PrintWriter`. Nada mais de exceções.
- **Outros:** `null` e comentários.
- **Imports permitidos:** `java.util.Random`, `java.io.PrintWriter`, `java.io.IOException`. Nenhum outro.

## 3.3 Recursos proibidos

- **Exceções:** `try`, `catch`, `finally`, `throw` e `throws` (exceto o caso autorizado em 3.2).
- **Tipos abstratos e orientação a objetos:** `abstract`, `interface`, `extends`, `implements`, genéricos (`<T>`, `<Integer>`), `enum`, `record`, qualquer classe além de `Trabalho2`, classes internas, classes anônimas e `package`.
- **Coleções e utilitários prontos:** `ArrayList`, `LinkedList`, `List`, `Map`, `HashMap`, `Set`, `HashSet`, `Arrays` (inclusive `Arrays.fill`, `Arrays.sort`, `Arrays.copyOf`), `Collections`, `Math` (inclusive `Math.abs`, `Math.max`, `Math.pow`), `StringBuilder`, `Scanner`, `Integer`, `Float`, `Long` (inclusive `Integer.parseInt` e `Integer.MAX_VALUE`), `System.arraycopy`, `clone()`, `hashCode()`, `equals()`, `compareTo()`, métodos de `String` (exceto `String.format`), streams, lambdas (`->`) e referências de método (`::`).
- **Comandos que simplificam o código:** operador ternário (`? :`), `switch`, `break`, `continue`, `for` de iteração direta (`for (int x : vetor)`), `var` e o modificador `final`.
- **Tipos não autorizados:** `double`, `char`, `byte`, `short`.
- **Bibliotecas de terceiros, frameworks de teste (inclusive JUnit) e geradores de código.**

### 3.4 Nomes em português

- Todo identificador criado pelo grupo (variável, parâmetro, vetor, constante e método) deve estar em português, sem acentos nem cedilha. Exemplos: `posicao`, `colisoes`, `tabela`, `chaves`, `quantidade`.
- Contadores de laço também: use `indice`, `posicao` ou `contador`. **Nomes de uma letra (`i`, `j`, `k`, `n`, `m`, `x`) e nomes em inglês (`index`, `count`, `table`, `key`, `temp`, `next`) são proibidos.**
- Padrão de escrita: variáveis e métodos em `camelCase` (`acessosInsercao`), constantes em MAIÚSCULAS (`VAZIO`).
- Ficam fora desta regra as palavras reservadas e os nomes da API do Java (`main`, `args`, `String`, `System`, `Random`, `nextInt`, `nanoTime`, `PrintWriter`, `length`) e os nomes prescritos neste documento.
- Comentários em português. Cada método começa com um comentário dizendo o que faz e o que retorna.

### 3.5 Semente do sorteio (obrigatória)

- Todo sorteio usa `Random sorteio = new Random(1990);`, criado uma única vez por conjunto de chaves. A ordem e a quantidade de chamadas a `nextInt` devem ser exatamente as descritas neste documento: só assim os números do seu programa coincidem com os do gabarito.
- Proibido: `new Random()` sem semente, qualquer outra semente, `Math.random()`, relógio como semente e qualquer outro gerador.
- Contagens (posições usadas, colisões, acessos etc.) devem ser **idênticas** às do gabarito. Valores de tempo variam entre computadores e não são validados.

### 3.6 Compilação e execução

- JDK 17 ou superior. O comando `javac Trabalho2.java` deve compilar sem erros, e `java Trabalho2` deve executar sem argumentos e sem nenhuma digitação.
- Um único arquivo `Trabalho2.java`, sem `package`, contendo somente a classe `Trabalho2`.
- O programa deve terminar em até 60 segundos e gravar `resultados_trabalho2.csv` na pasta onde foi executado.
- Se o arquivo não compilar, os critérios 1 a 5 da rubrica (seção 9) recebem 0.

### 3.7 Grupos

- O trabalho é feito em grupo de **3 ou 4 acadêmicos**. Grupos com 1, 2 ou mais de 4 integrantes não são aceitos.
- **Não é permitida a realização individual da atividade.** Entrega feita por um único acadêmico, ou por grupo fora do tamanho permitido, não é aceita e a atividade recebe nota 0 (zero).
- O trabalho de cada grupo é o que for sorteado em sala.
- O nome completo e a matrícula de todos os integrantes constam no cabeçalho e na declaração do relatório (item 7.1).
- Todos os integrantes devem saber explicar qualquer parte do código e do relatório (item 3.1).

## 4. Especificação técnica

Todos os nomes, assinaturas, fórmulas e algoritmos desta seção são **obrigatórios**. O grupo pode criar métodos auxiliares extras, desde que com nomes em português e dentro das regras da seção 3.

## 4.1 Variáveis estáticas obrigatórias

Declaradas dentro de `Trabalho2`, fora de qualquer método:

```
static int VAZIO = -1;                // marca de posição livre
static int TAMANHO = 1000;            // número de posições da tabela
static int QUANTIDADE = 700;          // chaves por conjunto
static int acessos = 0;               // contador de acessos (item 4.5)
static int colisoes = 0;              // contador de colisões (item 4.5)
static int duplicadasDescartadas = 0; // contador do gerador de chaves (item 5.2)
```

## 4.2 As quatro funções hash

Todas recebem uma chave de 7 dígitos (1.000.000 a 9.999.999) e devolvem uma posição entre 0 e 999. A tabela tem sempre 1000 posições.

| Nº | Método | Fórmula exata | Exemplo com a chave 9837561 |
|---|---|---|---|
| 1 | `hashDivisao(int chave)` | `chave % 1000` (os três últimos dígitos, como nos slides) | 561 |
| 2 | `hashPrimo(int chave)` | `chave % 997` (997 é o maior primo menor que 1000) | 162 |
| 3 | `hashDobra(int chave)` | Três partes: `parteFinal = chave % 1000`, `parteMeio = (chave / 1000) % 1000`, `parteInicio = chave / 1000000`. Retorna `(parteFinal + parteMeio + parteInicio) % 1000` | 561 + 837 + 9 = 1407, que dá 407 |
| 4 | `hashMeioQuadrado(int chave)` | `long quadrado = (long) chave * chave;` e retorna `(int) ((quadrado / 100000) % 1000)`, isto é, os dígitos nas posições 6, 7 e 8 do quadrado, contando da direita (a unidade é a posição 1) | 64 |

Observações obrigatórias:

- A função 2 **nunca** devolve 997, 998 ou 999, porque o resto da divisão por 997 vai de 0 a 996. Isso faz parte do experimento: as três posições só podem ser ocupadas por rehashing.
- Na função 4, o `long` é obrigatório. A conversão `(long)` vem **antes** da multiplicação. Escrever `chave * chave` em `int` estoura o limite do tipo e dá resultado errado.
- As funções 3 e 4 usam só `int` e `long`; não há `Math`.

## 4.3 Assinaturas obrigatórias

| Método | Retorno | Comportamento |
|---|---|---|
| `static int hashDivisao(int chave)` | posição | Item 4.2. |
| `static int hashPrimo(int chave)` | posição | Item 4.2. |

| Método | Retorno | Comportamento |
|---|---|---|
| `static int hashDobra(int chave)` | posição | Item 4.2. |
| `static int hashMeioQuadrado(int chave)` | posição | Item 4.2. |
| `static int aplicaHash(int funcao, int chave)` | posição | `funcao` vale 1, 2, 3 ou 4 e chama o método correspondente. Usa `if`, `else if` e `else` (sem `switch`; o `else` final trata o 4). |
| `static String nomeFuncao(int funcao)` | texto | Devolve `"divisao"`, `"primo"`, `"dobra"` ou `"meioQuadrado"` para 1, 2, 3 ou 4. |
| `static String nomeConjunto(int conjunto)` | texto | Devolve `"A"`, `"B"`, `"C"` ou `"D"` para 1, 2, 3 ou 4. |
| `static int rehash(int posicao)` | posição | Retorna `(posicao + 1) % TAMANHO`. |
| `static int[] criaTabela()` | vetor | Cria `int[TAMANHO]` e grava `VAZIO` em todas as posições com `for`. |
| `static int insere(int[] tabela, int funcao, int chave)` | 1, 0 ou -1 | Algoritmo 4.4.1. |
| `static int busca(int[] tabela, int funcao, int chave)` | posição ou -1 | Algoritmo 4.4.2. |
| `static int[] geraConjunto(Random sorteio, int conjunto, int quantidade)` | vetor | Item 5.2. |
| `static void executaCombinacao(PrintWriter arquivo, int conjunto, int[] chaves, int funcao)` | nenhum | Item 5.3. |
| `static void executaTestes()` | nenhum | Seção 6. |
| `public static void main(String[] args) throws IOException` | nenhum | Chama `executaTestes()`, cria o `PrintWriter`, grava o cabeçalho e, para cada conjunto de 1 a 4, cria `new Random(1990)`, chama `geraConjunto` e depois `executaCombinacao` para as funções 1 a 4. |

## 4.4 Algoritmos exatos (rehashing linear)

Chaves são sempre inteiros maiores que zero.

### 4.4.1 `insere(tabela, funcao, chave)`

1. `posicao = aplicaHash(funcao, chave)`; `passos = 0`.
2. Enquanto `passos < TAMANHO`, repita:
3. `acessos++` e `passos++` (uma posição foi examinada).
4. Se `tabela[posicao] == VAZIO`: grave `chave` em `tabela[posicao]` e **retorne 1**.
5. Se `passos == 1` (a posição inicial já estava ocupada): `colisoes++`.
6. Se `tabela[posicao] == chave`: **retorne 0** (duplicada; nada é alterado).
7. `posicao = rehash(posicao)`.
8. Se o laço terminou sem retorno, a tabela está cheia: **retorne -1**.

### 4.4.2 `busca(tabela, funcao, chave)`

1. `posicao = aplicaHash(funcao, chave)`; `passos = 0`.

2. Enquanto `passos < TAMANHO`, repita:
3. `acessos++` e `passos++`.
4. Se `tabela[posicao] == VAZIO`: **retorne -1**.
5. Se `tabela[posicao] == chave`: **retorne `posicao`**.
6. `posicao = rehash(posicao)`.
7. Se o laço terminou sem retorno: **retorne -1**.

### 4.5 Regra de contagem de acessos e de colisões

- **Acesso** = cada leitura de uma posição do vetor `tabela` para compará-la com `VAZIO` ou com a chave da operação. Os algoritmos 4.4.1 e 4.4.2 já indicam onde somar.
- **Colisão** = inserção em que a posição `aplicaHash(funcao, chave)` já estava ocupada no momento da chamada. É contada uma vez por chamada, qualquer que seja o retorno.
- Quem chama os métodos zera `acessos` e `colisoes` antes de cada bloco de medição e lê os valores depois. Os métodos nunca zeram esses contadores.

## 5. Conjuntos de chaves, experimento e saída

### 5.1 Os quatro conjuntos (700 chaves cada)

| Conjunto | Descrição | Fórmula de cada candidata | Faixa |
|---|---|---|---|
| A | Aleatório | `1000000 + sorteio.nextInt(9000000)` | 1.000.000 a 9.999.999 |
| B | Múltiplos de 1000 (terminam em 000) | `(1000 + sorteio.nextInt(9000)) * 1000` | 1.000.000 a 9.999.000 |
| C | Múltiplos de 10 (terminam em 0) | `(100000 + sorteio.nextInt(900000)) * 10` | 1.000.000 a 9.999.990 |
| D | Sequencial | `1000001 + gerados` (sem sorteio) | 1.000.001 a 1.000.700 |

Os conjuntos B, C e D imitam códigos reais: B e C são códigos que terminam com zeros (lotes, filiais) e D é uma numeração sequencial de produtos.

### 5.2 Geração: `geraConjunto(sorteio, conjunto, quantidade)`

1. Crie o vetor `chaves` com `quantidade` posições; `gerados = 0`; `duplicadasDescartadas = 0`.
2. Enquanto `gerados < quantidade`, calcule `candidata` pela fórmula do conjunto (tabela 5.1). Nos conjuntos A, B e C há **uma única chamada a `nextInt` por candidata**; o conjunto D não chama `nextInt`.
3. Compare `candidata` com `chaves[0]` até `chaves[gerados - 1]` usando `for`. Se já existir, faça `duplicadasDescartadas++` e descarte. Se não existir, grave em `chaves[gerados]` e faça `gerados++`.
4. Retorne `chaves`.

O `main` cria `new Random(1990)` **novamente para cada conjunto**, antes de chamar `geraConjunto`, inclusive para o conjunto D. As mesmas 700 chaves de um conjunto são usadas nas quatro funções.

### 5.3 Roteiro de `executaCombinacao(arquivo, conjunto, chaves, funcao)`

1. Crie o vetor `contagem` com `TAMANHO` posições, todas 0. Para cada chave, em ordem, faça `contagem[aplicaHash(funcao, chave)]++`. Essa contagem é feita **antes** de qualquer inserção e não envolve rehashing.
2. Percorra `contagem` com `for` e calcule `usadas` (posições com contagem maior que 0), `maiorConcentracao` (maior valor) e `somaQuadrados` (soma de `contagem[posicao] * contagem[posicao]`, em `int`).
3. Crie a tabela, zere `acessos` e `colisoes`, faça `maiorAcessos = 0` e guarde `inicio = System.nanoTime()`. Insira `chaves[0]`, `chaves[1]`, ... nessa ordem; a cada inserção, calcule os acessos gastos nela (`acessos` depois menos antes) e atualize `maiorAcessos`. Guarde `fim = System.nanoTime()`, `tempoInsercao = (fim - inicio) / quantidade` (`long`), `colisoes` e `acessos`.
4. Zere `acessos` e busque `chaves[0]`, `chaves[1]`, ... nessa ordem. Guarde o total de acessos.
5. Monte a linha (item 5.5), imprima no console e grave no arquivo.

### 5.4 Colunas da saída (12, nesta ordem)

| Coluna | Definição |
|---|---|
| `conjunto` | `A`, `B`, `C` ou `D`. |
| `funcao` | `divisao`, `primo`, `dobra` ou `meioQuadrado`. |
| `chaves` | 700. |
| `posicoesUsadas` | Quantas posições receberam pelo menos 1 chave da função hash (antes do rehashing). |
| `posicoesNuncaUsadas` | `TAMANHO - posicoesUsadas`. |
| `maiorConcentracao` | Maior número de chaves que a função hash mandou para uma mesma posição. |
| `somaQuadrados` | Soma, nas 1000 posições, do quadrado da quantidade de chaves da posição. Quanto menor, mais uniforme a distribuição (o mínimo possível com 700 chaves é 700). |
| `colisoes` | Valor de `colisoes` após as 700 inserções. |
| `acessosInsercaoTotal` | Valor de `acessos` após as 700 inserções. |
| `maiorAcessosUmaInsercao` | Maior número de acessos gasto por uma única inserção. |
| `acessosBuscaSucessoMedia` | `(float) total de acessos das 700 buscas / quantidade`. |
| `tempoInsercaoNsPorOp` | `tempoInsercao` do roteiro (nanossegundos por inserção). |

A média é calculada com `float` na forma `(float) soma / quantidade`, sem outra ordem de operações, para que o arredondamento seja igual ao do gabarito.

### 5.5 Formato da saída

- A saída do console e o arquivo `resultados_trabalho2.csv` contêm **as mesmas 17 linhas**: 1 de cabeçalho e 16 de dados (4 conjuntos × 4 funções).
- Ordem das linhas: conjunto A, B, C, D; dentro de cada conjunto, funções 1, 2, 3, 4.
- Separador de campos: ponto e vírgula, sem espaços. Cada linha de dados é montada com `String.format` e este formato exato:

```
"%s;%s;%d;%d;%d;%d;%d;%d;%d;%.2f;%d"
```

- Cabeçalho, em uma única linha:

```
conjunto;funcao;chaves;posicoesUsadas;posicoesNuncaUsadas;maiorConcentracao;somaQuadrados;colisoes;acessosInsercaoTotal;maiorAcessosUmaInsercao;acessosBuscaSucessoMedia;tempoInsercaoNsPorOp
```

- O separador decimal (vírgula ou ponto) depende da configuração regional do computador. Qualquer um é aceito, desde que o arquivo seja aberto corretamente na planilha.
- Ordem de tudo que o programa imprime: (1) resultado dos testes da seção 6; (2) cabeçalho; (3) as 16 linhas de dados. Nenhuma outra linha.
- **Conferência obrigatória:** em todas as linhas, `acessosBuscaSucessoMedia` é igual a `acessosInsercaoTotal ÷ chaves` (arredondado a 2 casas), porque, sem remoções, a busca percorre o mesmo caminho da inserção. Se não for, há erro no código.

# 6. Testes de validação

`executaTestes()` roda os 17 testes abaixo, nesta ordem, antes do experimento. Cada teste compara o resultado do código com o valor esperado usando `if`. A saída é exatamente uma linha por teste, no formato `TESTE <código>: APROVADO` ou `TESTE <código>: REPROVADO`, e uma linha final `TESTES: <aprovados> de <total> aprovados`. Um programa correto imprime `TESTES: 17 de 17 aprovados`.

Regra comum: antes de cada teste que confere `acessos` ou `colisoes`, esses contadores são zerados pelo próprio teste.

## 6.1 Funções hash (H1 a H7)

Cada teste chama os quatro métodos de função hash com a mesma chave e confere os quatro resultados.

| Código | Chave | `hashDivisao` | `hashPrimo` | `hashDobra` | `hashMeioQuadrado` |
|---|---|---|---|---|---|
| H1 | 9837561 | 561 | 162 | 407 | 64 |
| H2 | 1234567 | 567 | 281 | 802 | 556 |
| H3 | 9999999 | 999 | 89 | 7 | 800 |
| H4 | 1000000 | 0 | 9 | 1 | 0 |
| H5 | 5654396 | 396 | 409 | 55 | 941 |
| H6 | 4967000 | 0 | 943 | 971 | 890 |

**H7:** `geraConjunto(new Random(1990), 1, 700)` e, para cada uma das 700 chaves e cada uma das 4 funções (via `aplicaHash`), o resultado fica entre 0 e 999.

H3 é o teste do `long`: com `int` o quadrado de 9999999 estoura e o resultado deixa de ser 800.

## 6.2 Inserção e busca (I1 a I6)

Todos usam a função 1 (divisão). As chaves 5001, 7001 e 9001 resultam na posição 1.

| Código | Preparação | Resultado esperado |
|---|---|---|
| I1 | Tabela nova; `acessos` e `colisoes` zerados; inserir 5001, 7001 e 9001 | `tabela[1] == 5001`, `tabela[2] == 7001`, `tabela[3] == 9001` |
| I2 | Após I1 | `colisoes == 2` e `acessos == 6` |
| I3 | `busca(tabela, 1, 9001)` | Retorna 3; `acessos == 3` |
| I4 | `busca(tabela, 1, 4001)` (também posição 1, ausente) | Retorna -1; `acessos == 4` |
| I5 | Tabela nova; inserir 1000999 e depois 2000999 (ambas na posição 999) | `tabela[999] == 1000999` e `tabela[0] == 2000999` (volta ao início) |

| Código | Preparação | Resultado esperado |
|---|---|---|
| I6 | Após I5; `insere(tabela, 1, 2000999)` de novo | Retorna 0; `acessos == 2` |

### 6.3 Conjuntos de chaves (S1 a S4)

Cada teste cria `new Random(1990)` e chama `geraConjunto` com 700 chaves. O conjunto D não depende do sorteio: qualquer `sorteio` serve, pois esse conjunto não chama nextInt.

| Código | Conjunto | Três primeiras chaves | Última chave (`chaves[699]`) | `duplicadasDescartadas` |
|---|---|---|---|---|
| S1 | A | 4429279, 3976163, 9187615 | 9691227 | 0 |
| S2 | B | 1279000, 7163000, 7615000 | 7494000 | 32 |
| S3 | C | 8292790, 3761630, 1876150 | 6289620 | 0 |
| S4 | D | 1000001, 1000002, 1000003 | 1000700 | 0 |

### 6.4 Linhas de conferência do experimento

Depois de rodar o programa completo, as colunas determinísticas (todas, menos `tempoInsercaoNsPorOp`) das três linhas abaixo devem ser **idênticas** às do seu CSV. O separador decimal pode aparecer como vírgula, conforme a configuração regional do computador.

```
A;dobra;700;502;498;5;1198;250;1641;40;2.34
B;divisao;700;1;999;700;490000;699;245350;700;350.50
D;meioQuadrado;700;250;750;7;2822;650;4392;10;6.27
```

A linha `B;divisao` se confere à mão: as 700 chaves vão para a posição 0, então a inserção de ordem  gasta acessos e o total é 700 × 701 ÷ 2 = 245350. As outras 13 linhas são conferidas pelo professor.

## 7. Relatório

Arquivo `relatorio_trabalho2.pdf`, em português, com no máximo 5 páginas A4 (fonte de 11 ou 12 pontos, margens de pelo menos 2 cm). Todo o texto é escrito pelos integrantes do grupo, sem IA generativa (item 3.1). Os gráficos podem ser feitos em planilha (Excel, LibreOffice Calc ou Google Planilhas) a partir do CSV.

### 7.1 Estrutura obrigatória, nesta ordem

1. **Cabeçalho:** título do trabalho, disciplina, turma, nome completo e matrícula de cada integrante, data.
2. **Implementação (até 1 página):** como cada função hash foi implementada, decisões tomadas, dificuldades encontradas e o resultado da linha `TESTES: <aprovados> de <total> aprovados` do programa.
3. **Tabela de resultados:** as 16 linhas do CSV com as colunas `conjunto`, `funcao`, `posicoesUsadas`, `maiorConcentracao`, `somaQuadrados`, `colisoes`, `acessosInsercaoTotal`, `maiorAcessosUmaInsercao` e `acessosBuscaSucessoMedia`.

4. **Três gráficos de barras agrupadas**, todos com título, eixos nomeados com unidade, legenda, um grupo de barras por conjunto (A, B, C, D) e uma barra por função:
5. `acessosBuscaSucessoMedia`, com escala logarítmica no eixo vertical (os valores variam de 1 a mais de 350).
6. `posicoesUsadas`.
7. `maiorConcentracao`.
8. **Respostas às perguntas do item 7.2.**
9. **Conclusão (até 10 linhas):** qual função o grupo escolheria como padrão e em que condições escolheria outra.
10. **Declaração:** "Declaramos que o código e o texto deste trabalho foram produzidos pelos integrantes do grupo, sem uso de ferramenta de inteligência artificial generativa e sem cópia de outros grupos ou de fontes externas." Seguida do nome e da matrícula de cada integrante.
11. **Referências:** slides da disciplina e qualquer outro material consultado.

### 7.2 Perguntas obrigatórias

Cada resposta tem de 3 a 8 linhas e cita números da tabela de resultados.

1. No conjunto A (aleatório), as quatro funções tiveram desempenho parecido? Qual teve menos colisões e por que a diferença entre elas é pequena?
2. No conjunto B, o que aconteceu com a função divisão? Explique por que `posicoesUsadas` foi 1 e quantos acessos gastou a inserção da última chave.
3. Ainda no conjunto B, por que a função primo se saiu tão melhor que a divisão? Relacione com o fato de $1000 = 2^3 \times 5^3$ e de 997 ser primo.
4. No conjunto C, por que a função divisão usa apenas 100 posições? Que fração da tabela ficou sem nenhuma chave vinda da função hash?
5. No conjunto D (sequencial), a divisão, o primo e a dobra não tiveram nenhuma colisão. Explique por quê. Por que o meio do quadrado foi pior? Cite `posicoesUsadas` e `acessosBuscaSucessoMedia`.
6. Explique por que `posicoesNuncaUsadas` da função primo é sempre pelo menos 3.
7. Mostre, com a conta, por que a função do meio do quadrado precisa de `long` para a chave 9999999. Qual é o maior valor que cabe em um `int` em Java?
8. Em qual conjunto foi maior a diferença entre a melhor e a pior função? Compare com `acessosBuscaSucessoMedia` e com `somaQuadrados` e explique o que `somaQuadrados` mede.
9. Por que `maiorAcessosUmaInsercao` pode ser maior que `maiorConcentracao`? Cite uma linha do CSV em que isso acontece.
10. Se os códigos dos produtos do comerciante dos slides terminassem sempre em 00 (por exemplo, 1234500), qual função o grupo recomendaria e por quê?

## 8. Entrega

O grupo entrega **exatamente três arquivos**, com estes nomes (minúsculas, sem acentos, sem espaços):

| Arquivo | Conteúdo |
|---|---|
| `Trabalho2.java` | Código-fonte único, com a classe `Trabalho2`. |
| `resultados_trabalho2.csv` | CSV gerado pela execução do programa entregue, sem nenhuma edição manual. |
| `relatorio_trabalho2.pdf` | Relatório da seção 7. |

- Os três arquivos são enviados dentro de um único arquivo `trabalho2_grupoNN.zip`, em que `NN` é o número do grupo com dois dígitos (exemplo: `trabalho2_grupo07.zip`).
- Plataforma de entrega: github público .Basta um integrante enviar, em nome do grupo.
- Vale o **último envio** feito. Se houver envio depois do prazo, a hora desse último envio é a usada para calcular o atraso (seção 10).
- Não entregar arquivos `.class`, pastas de IDE, projetos compactados ou qualquer outro arquivo.
- O professor compila e executa o `Trabalho2.java` entregue e compara a saída com o gabarito. Colunas determinísticas diferentes do gabarito são erro de implementação, mesmo que o CSV entregue esteja "certo".

## 9. Critérios de avaliação

A nota do trabalho é a soma dos critérios abaixo (0 a 100), menos o desconto por atraso da seção 10, com mínimo 0.

| # | Critério | Pontos | Como é avaliado |
|---|---|---|---|
| 1 | As quatro funções hash (4.2) | 20 | 5 pontos por função: fórmula idêntica à especificação, inclusive o `long` no meio do quadrado. |
| 2 | Inserção e busca com rehashing (4.4) | 10 | Algoritmos, retornos, contagem de `acessos` e de `colisoes` idênticos à especificação. |
| 3 | Geração dos conjuntos e semente (5.2 e 3.5) | 10 | `Random(1990)` por conjunto, fórmulas A a D, ordem das chamadas, descarte de duplicadas. |
| 4 | Experimento e saída (5.3 a 5.5) | 15 | 16 combinações, 17 linhas, cabeçalho, formato e CSV; colunas determinísticas iguais ao gabarito. |
| 5 | Testes (seção 6) | 10 | 10 × (testes corretamente implementados e aprovados ÷ 17). |
| 6 | Relatório: tabela e gráficos (7.1, itens 3 e 4) | 10 | Tabela completa e 3 gráficos com título, eixos, unidades, legenda e a escala pedida. |
| 7 | Relatório: perguntas e conclusão (7.2 e 7.1, item 6) | 15 | 1,2 ponto por pergunta respondida com números e correta (12 pontos) + conclusão (3 pontos). |
| 8 | Restrições de linguagem (3.2 e 3.3) | 5 | Começa em 5; perde 1 ponto por categoria proibida encontrada (cada linha da lista 3.3 é uma categoria; conta uma vez, mesmo repetida). Mínimo 0. |
| 9 | Nomes em português e comentários (3.4) | 5 | Nomes: 3 pontos, perde 1 por nome inválido distinto (inglês ou de uma letra), |

| # | Critério | Pontos | Como é avaliado |
|---|---|---|---|
| | | | mínimo 0. Comentários: 2 pontos se todos os métodos têm o comentário inicial, 1 se mais da metade, 0 nos demais casos. |
| | **Total** | **100** | |

Relatório ausente zera os critérios 6 e 7. CSV ausente zera o critério 4. Se o `Trabalho2.java` não compilar, os critérios 1 a 5 recebem 0 (item 3.6).

# 10. Prazo, atraso e penalidades

O prazo final é o da seção 1, no horário de Brasília, e vale o horário de recebimento registrado pela plataforma de entrega.

## 10.1 Desconto por atraso

O atraso é contado em blocos de 24 horas corridas a partir do prazo final (sábados, domingos e feriados contam). O desconto é um percentual do **total do trabalho (100 pontos)**, subtraído da nota obtida:

| Atraso | Desconto | Pontos subtraídos |
|---|---|---|
| De 1 segundo até 24 horas | 35% | 35 |
| Mais de 24 horas até 48 horas | 50% | 50 |
| Mais de 48 horas até 72 horas | 70% | 70 |
| Mais de 72 horas | Trabalho zerado | Nota final 0 |

- Os descontos **não se acumulam**: vale apenas o da faixa em que o envio caiu.
- A nota final nunca fica abaixo de 0.
- Exemplo 1: nota obtida 82, envio 30 horas depois do prazo (faixa de 50%): 82 − 50 = 32.
- Exemplo 2: nota obtida 40, envio 10 horas depois do prazo (faixa de 35%): 40 − 35 = 5.
- Exemplo 3: envio 80 horas depois do prazo: nota 0, qualquer que seja a qualidade do trabalho.
- Problemas de internet, de computador ou da plataforma não prorrogam o prazo. Envie com antecedência.

## 10.2 Resumo das penalidades

| Situação | Consequência |
|---|---|
| Uso de IA generativa em qualquer parte | Nota 0 para todos os integrantes do grupo. |
| Cópia de outro grupo, de repositório ou de site | Nota 0 para todos os grupos envolvidos. |
| Trabalho individual, ou grupo com menos de 3 ou mais de 4 integrantes (item 3.7) | Trabalho não aceito: nota 0. |
| Integrante que não consegue explicar o próprio trabalho | Tratado como indício de IA ou de cópia (item 3.1). |
| Atraso de até 24 h, 48 h ou 72 h | Desconto de 35, 50 ou 70 pontos. |
| Atraso de mais de 72 h | Nota 0. |
| Código que não compila | Critérios 1 a 5 da seção 9 recebem 0. |
| Recurso proibido no código | Perde 1 ponto por categoria no critério 8. |
| Nomes fora do padrão ou falta de comentários | Perda de pontos no critério 9. |

A penalidade por IA ou cópia vale independentemente do prazo e dos demais descontos.